

Modul 5, Pengujian Perisian & Automasi Ujian

Module 5, Software Testing & Test Automation

Laksanakan pengujian yang berkesan, fungsian & bukan fungsian, dan fahami ke mana automasi harus diletakkan dalam saluran penyampaian anda.

Execute effective testing, functional & non-functional, and understand where automation belongs in your delivery pipeline.

1j 30m / 1h30m

2 Makmal / Live Labs

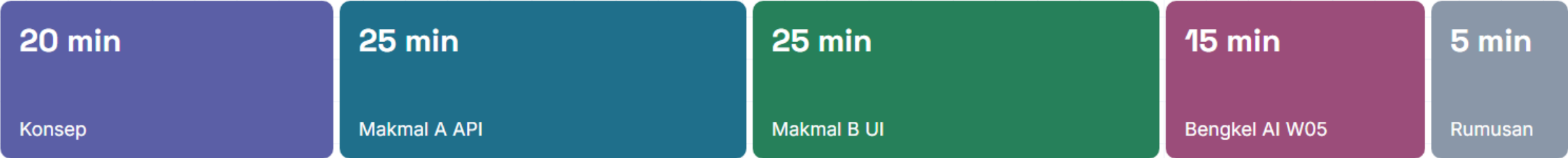
Projek: ShopFast

KRISA 2.0 · Bab 5 & 8

Jadual slot: Modul 5: Pengujian Perisian dan Automasi Ujian

Run sheet / session timing

Hari 2, 09:00 hingga 10:30 · 90 minit · kira-kira 80% amali



Masa	Segmen	Tempoh
09:00 hingga 09:20	Konsep	20 minit
09:20 hingga 09:45	Makmal A API	25 minit
09:45 hingga 10:10	Makmal B UI	25 minit
10:10 hingga 10:25	Bengkel AI W05	15 minit
10:25 hingga 10:30	Rumusan	5 minit

Modul 5 membuka Hari 2, di dalam fasa SAH (VERIFY)

Hari 1 menetapkan piawaian (Rancang) & mencegah kecacatan awal (Bina-Dalam). Kini kita mengesahkan ShopFast benar-benar berfungsi seperti dirancang, dengan bukti, bukan andaian.

Day 1 set the standards (Plan) & prevented early defects (Build-in). Now we verify ShopFast truly works as planned, with evidence, not assumptions.



RTM

Setiap kes ujian pagi ini terus dipetakan ke RTM v0 dari Modul 2, keterjejukan tidak putus. / Every test case maps back to the RTM v0 from Module 2, traceability stays unbroken.

Daripada “ia nampak berfungsi” kepada bukti automatik

Executing effective testing (functional & non-functional) and understanding automation strategy, so every ShopFast change can be verified in seconds, not days.

1h30

DURASI / DURATION

2

MAKMAL / LABS (API + UI)

5

ALATAN / INDUSTRY TOOLS

2

HASIL KERJA /
DELIVERABLES



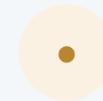
Jenis Ujian / Test Types

Smoke, Sanity, Regression, Exploratory



Bukan Fungsian / Non-Functional

Performance, Security, Accessibility



Automasi / Automation

Piramid Automasi + integrasi CI/CD / Automation
Pyramid + CI/CD

Empat lensa pengujian fungsian / Four lenses of functional testing

Pilih lensa yang betul mengikut situasi, build baharu, pembetulan pepijat, sebelum release, atau meneroka. / Pick the right lens for the moment.

Smoke

"Bolehkah app dimuatkan?" Semakan pantas & cetek selepas build baharu.
"Does the build even start?" A fast, shallow check right after a new build.

Sanity

Fokus sempit selepas satu pembetulan, sahkan satu ciri waras tanpa regresi penuh.
A narrow check after one fix, confirm one feature is sane without full regression.

Regression

"Adakah ciri lama masih berfungsi?" Larian semula suite sedia ada selepas perubahan.
"Do old features still work?" Re-run the existing suite after any change.

Exploratory

Tiada skrip. Penguji meneroka aplikasi secara kreatif untuk menemui yang tak terfikir.
No script, the tester roams creatively to surface what scripts miss.

“Ia berfungsi” ≠ “Ia sedia produksi” / “Works” ≠ “Production-ready”

Pengujian bukan fungsian menyoal: pantas ke, selamat ke, boleh capai semua ke? / Non-functional testing asks: is it fast, safe, and accessible to everyone?



Prestasi / Performance

Load / Stress, berapa ramai pengguna serentak sebelum sistem merosot?
How many concurrent users before response time degrades?

[JMeter](#) [k6](#)



Keselamatan / Security

Imbas kelemahan OWASP pada API checkout sebelum penyerang menemuinya.
Scan the checkout API for OWASP vulnerabilities first.

[OWASP ZAP](#) [Burp Suite](#)



Kebolehcapaian / Accessibility

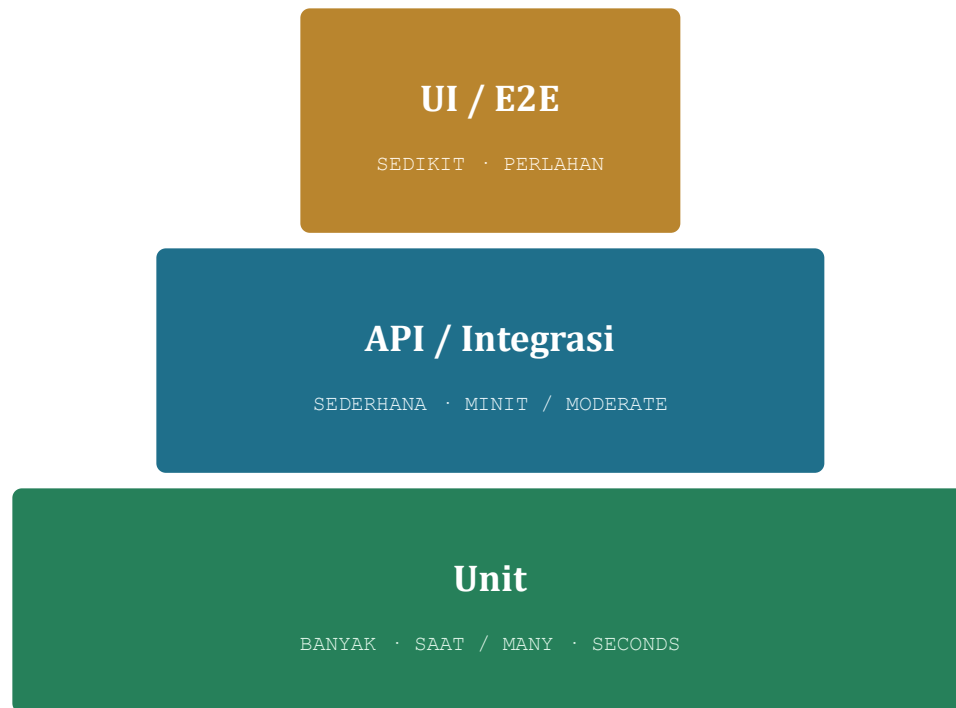
Pembaca skrin, kontras warna & navigasi papan kekunci untuk semua pengguna.
Screen reader, color contrast & keyboard nav for every user.

[WCAG 2.2](#) [axe DevTools](#)

NOTA / NOTE: Ketiga-tiganya didemo langsung, bukan untuk dikuasai penuh hari ini, tetapi supaya anda kenal bila memanggilnya. / All three are live-demoed, so you recognize when to reach for them.

Piramid Automasi, di mana masa & wang patut pergi / The Automation Pyramid

Lebih tinggi lapisan, lebih perlahan & mahal ujian itu. Strategi sihat: banyak di bawah, sedikit di atas. / Higher layer = slower & pricier. Healthy strategy: lots at the base, few at the top.



MAHAL / EXPENSIVE

Unit, Asas piramid / the base

Ratusan ujian, saat untuk berjalan, tanpa DB/rangkaian. Isolasi satu fungsi sahaja.
Hundreds of tests, seconds to run, no DB/network.

Integrasi / API, Lapisan tengah (Makmal A)

Puluhan ujian, minit untuk berjalan, cth. koleksi [Postman](#) untuk API checkout.
Dozens of tests, minutes to run (Lab A).

UI / E2E, Puncak (Makmal B)

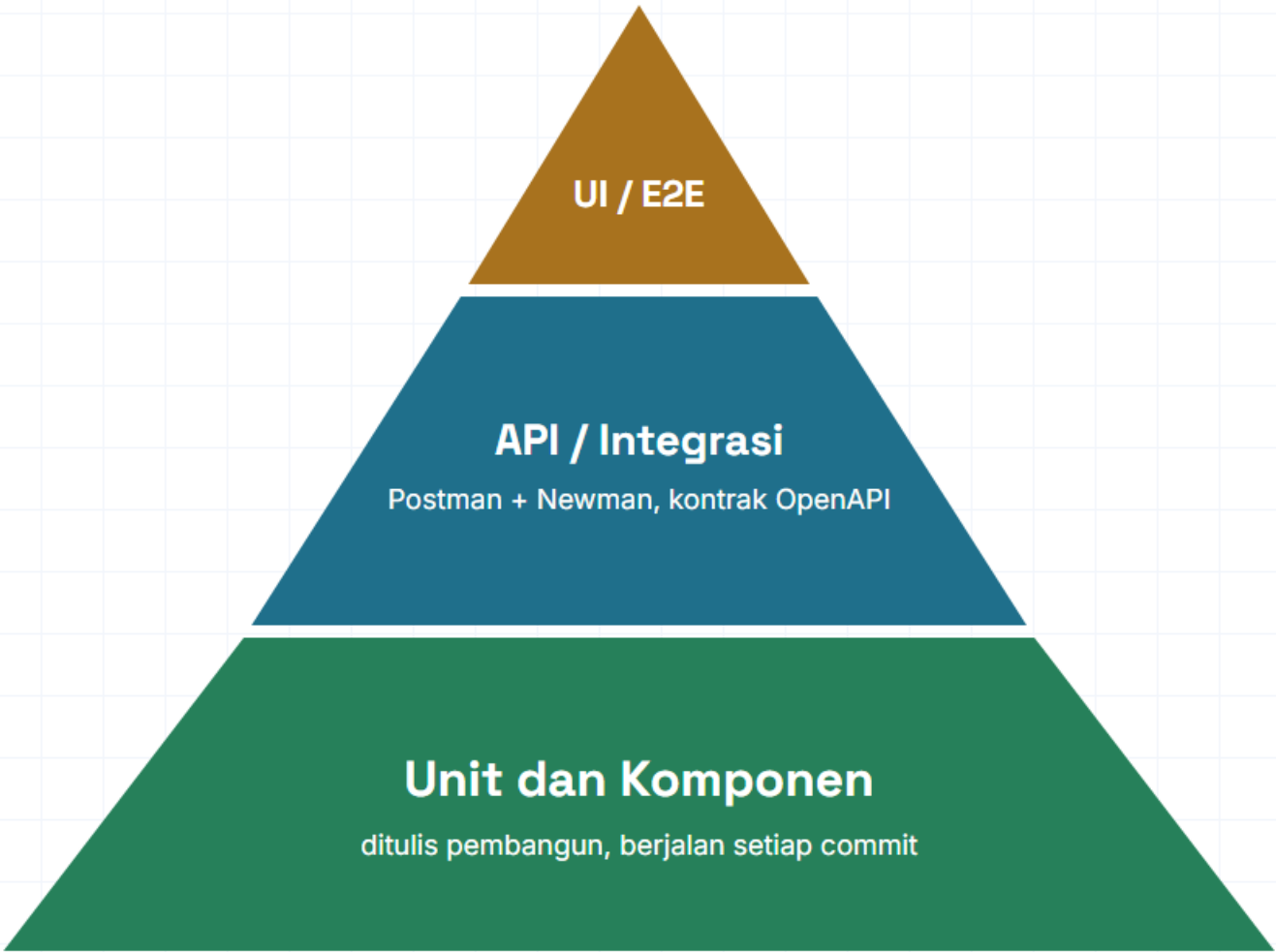
Sedikit ujian, paling perlahan & rapuh, simulasi pengguna sebenar log masuk to checkout.
Few tests, slowest & most brittle (Lab B).

MURAH / CHEAP ↓

Piramid automasi ujian: di mana masa dan kos patut pergi

Test automation pyramid: where time and money should go

Banyak ujian unit yang pantas di bawah, sedikit ujian UI yang perlahan di atas. Elak bentuk kon ais krim.



Lapisan	Bil. ujian	Masa	Contoh alat
UI / E2E	5 hingga 10%	minit	Playwright, Selenium
API	20 hingga 30%	saat	Postman, Newman, k6
Unit	60 hingga 70%	milisaat	Jest, pytest, JUnit

KRISAv2 Bab 7.10: ujian SIT, UAT dan prestasi dijalankan di persekitaran **staging**. Bab 8: automasi dalam saluran CI/CD sebelum Laporan Release.

Automasi bernilai bila ia berjalan sendiri / Automation pays off when it runs itself

Ujian yang anda tulis pagi ini menjadi Quality Gate dalam CI/CD, dilaksanakan setiap kali kod ditolak. / The tests you write become a Quality Gate, run on every push.



PRATONTON / PREVIEW: Modul 8 memasukkan tepat ujian yang anda tulis sekarang ke [GitHub Actions](#) sebagai Quality Gate produksi.

Quality Gate dalam CI/CD: tiada merge tanpa larian hijau

CI/CD Quality Gate: no merge without a green run

KRISAv2 Bab 7 7.9: konfigurasi CI/CD dan skrip automasi disimpan dalam repositori. Bab 8: automasi sebelum Laporan Release.



HIJAU

Semua ujian lulus, tiada severity 1 dan 2 terbuka, liputan RTM cukup. Merge dibenarkan dan artifak laporan disimpan.

MERAH

Mana-mana langkah gagal. Merge disekat, pepijat dilog (PPrISA14), pembangun baiki dan tolak semula.

```
lab/shopfast .github/workflows/shopfast-quality-gate.yml job: quality-gate
```

Testing and Automation

Writing automated tests that are trustworthy, running them continuously, and knowing where formal testing fits

01 What makes a good automated test

02 xUnit anatomy

03 Fatal vs non-fatal assertions

04 Fixtures and fresh state

05 Custom failure messages

06 Test-driven development

07 Continuous integration and build freeze

08 Dynamic analysis, formal runs, test goals

What makes a good automated test

Six properties from the Google Test primer, with equivalents in Playwright Test and pytest

Property	How Google Test supports it	Playwright Test / pytest equivalent
Independent and repeatable	Each test runs on a fresh fixture object; run one alone with <code>--gtest_filter</code>	Fresh browser context per test; run one test with <code>-g</code> (Playwright) or <code>-k</code> (pytest)
Well organised, mirrors the code	Related tests grouped into test suites that share fixtures	<code>test.describe</code> blocks and spec files; pytest modules and classes
Portable and reusable	Works across operating systems and compilers	Chromium, Firefox and WebKit projects; pytest runs wherever Python runs
Informative when it fails	Moves on to the next test; <code>EXPECT_*</code> reports several failures	<code>expect.soft</code> , traces and screenshots; pytest prints the compared values
No housekeeping	Tests register themselves; no list to maintain	Spec files and <code>test_</code> functions are discovered automatically
Fast	Expensive setup shared once per suite (<code>SetUpTestSuite</code>)	Worker-scoped fixtures and parallel workers; pytest fixture scopes

xUnit anatomy: the building blocks

Test program

One executable or run holding many suites: `RUN_ALL_TESTS()` in C++, `npx playwright test` for ShopFast.

Test suite

Related tests grouped to mirror the code. Older Google Test docs call it a test case; Playwright uses `test.describe`.

Test

One named scenario, such as `TEST(CartPricing, AddsShipping)` or `test('adds shipping', ...)`.

Assertion

A statement that checks a condition. Result: success, non-fatal failure or fatal failure.

Test fixture

Shared objects and setup reused by several tests: a `TEST_F` class, `beforeEach`, or a `pytest` fixture.

Verdict

A test fails if it crashes or any assertion fails, fatal or not. Otherwise it passes.

Fatal vs non-fatal assertions

Fatal: ASSERT_* and expect

- Records the failure and aborts the current function immediately
- Use when later steps would be meaningless or crash, e.g. item never added
- Google Test: ASSERT_EQ, ASSERT_TRUE; only in functions returning void
- Playwright Test: a failing expect throws and ends the test
- pytest: a plain assert ends that test

Non-fatal: EXPECT_* and soft

- Records the failure and lets the test continue
- The test is still marked as failed at the end
- Reports several wrong values in one run, saving rerun cycles
- Google Test: EXPECT_EQ, EXPECT_TRUE
- Playwright Test: `expect.soft(locator).toHaveText(...)`
- pytest: needs a plugin such as pytest-check

Guard preconditions with fatal checks, then verify the many output values with non-fatal checks.

Google Test: ASSERT then EXPECT

- `ASSERT_TRUE` guards the precondition: no item, no point checking totals
- `EXPECT_EQ` checks each figure and keeps going after a mismatch
- `<<` appends a custom message shown only when the check fails
- Money kept in sen (integers) to avoid floating point surprises
- Cart is an illustrative C++ port of the ShopFast pricing rules

CART_TEST.CC

```
#include <gtest/gtest.h>
#include "cart.h"

TEST(CartPricing, TwoSongkokIncludeShipping) {
    Cart cart;
    ASSERT_TRUE(cart.Add("P002", 3500, 2))
        << "Add P002 qty 2 rejected, totals meaningless";
    EXPECT_EQ(cart.SubtotalSen(), 7000) << "2 x RM 35.00";
    EXPECT_EQ(cart.ShippingSen(), 800) << "RM 8.00 below RM 200";
    EXPECT_EQ(cart.TotalSen(), 7800) << "screen shows RM 78.00";
}

TEST(CartPricing, RejectsQuantityZero) {
    Cart cart;
    EXPECT_FALSE(cart.Add("P002", 3500, 0)) << "qty 0 must fail";
    EXPECT_EQ(cart.ItemCount(), 0u) << "cart must stay empty";
}
```

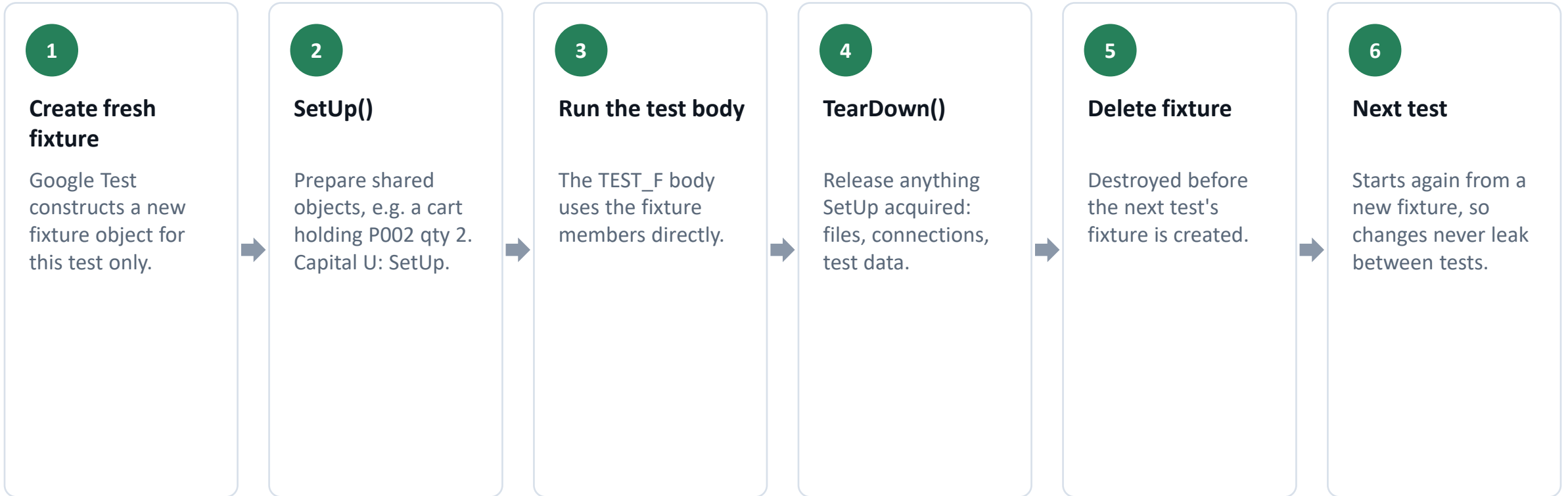
Playwright Test: expect then expect.soft

- Plain expect is fatal: the test stops if the item was not added
- expect.soft records each wrong figure and continues
- The second argument to expect is a custom failure message
- Locators use data-testid values, which survive layout changes
- RAYA15 is 15 percent of RM 70.00, so RM 10.50 off; total RM 67.50

TESTS/UI/PROMO.SPEC.JS

```
const { test, expect } = require('@playwright/test');
// beforeEach (shown on a later slide) has opened the page
test('RAYA15 on 2 x P002', async function ({ page }) {
  await page.getByTestId('qty-P002').fill('2');
  await page.getByTestId('add-P002').click();
  // Fatal: stop if the item never reached the cart
  await expect(page.getByTestId('cart-subtotal'),
    'P002 x 2 was not added').toHaveText('RM 70.00');
  await page.getByTestId('discount-input').fill('RAYA15');
  await page.getByTestId('discount-apply').click();
  // Soft: collect every wrong figure in one run
  await expect.soft(page.getByTestId('cart-discount'))
    .toHaveText('RM 10.50');
  await expect.soft(page.getByTestId('cart-shipping'))
    .toHaveText('RM 8.00');
  await expect.soft(page.getByTestId('cart-total'))
    .toHaveText('RM 67.50');
});
```

Fixtures: a fresh start for every test



Playwright equivalent: `test.beforeEach` and `test.afterEach`, with a new browser context and page for every test.

Playwright fixtures for ShopFast

- `beforeEach` runs before every test in the `describe` block
- Each test gets a new browser context, so a new session and empty cart
- Waiting for the cart API response avoids a load-time race
- `afterEach` keeps evidence only when a test did not pass
- The config option `screenshot: only-on-failure` does the same globally

TESTS/UI/CART.SPEC.JS

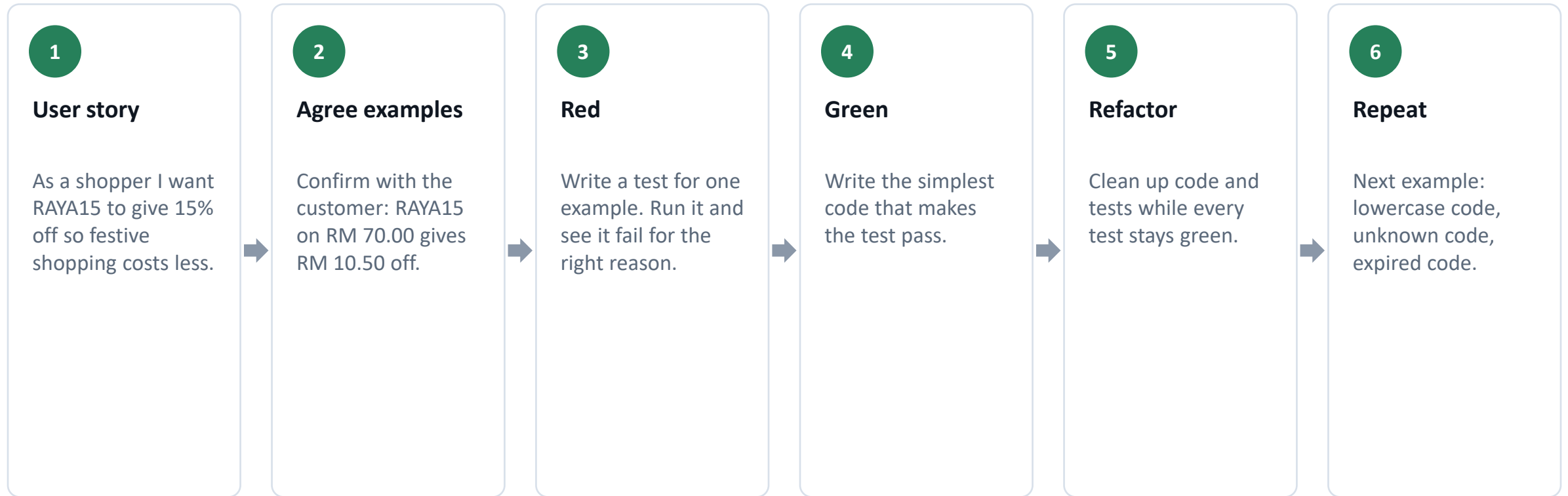
```
const { test, expect } = require('@playwright/test');
test.describe('ShopFast cart', function () {
  test.beforeEach(async function ({ page }) {
    const cartLoaded = page.waitForResponse('**/api/cart');
    await page.goto('/');
    await cartLoaded;
  });
  test.afterEach(async function ({ page }, testInfo) {
    if (testInfo.status !== testInfo.expectedStatus) {
      const shot = testInfo.outputPath('cart.png');
      await page.screenshot({ path: shot });
    }
  });
  test('empty cart for a new visitor', async function ({ page }) {
    const total = page.getByTestId('cart-total');
    await expect(total).toHaveText('RM 0.00');
  });
});
```

Custom failure messages that help

- ☐ State the expectation in business terms: cart-total should be RM 78.00 for 2 x P002
- ☐ Name the rule or requirement ID, for example REQ-CHK-03, so triage is quick
- ☐ Include the input that caused the failure: product, quantity, promo code
- ☐ In loops, include the index or item, as in the primer: differ at index i
- ☐ Google Test: `ASSERT_EQ(x.size(), y.size()) << "unequal length";`
- ☐ Playwright Test: `expect(locator, 'P002 x 2 was not added').toHaveText(...)`
- ☐ pytest: `assert total == expected, f"total for {qty} x P002"`
- ☐ Do not repeat what the framework already prints, such as actual and expected values

A good message lets someone who did not write the test understand the failure from the CI log alone.

Test-driven development from user stories



Red, green, refactor is Kent Beck's TDD cycle. The course notes start from user stories confirmed with the customer.

Continuous integration and build freeze

Continuous integration

Every push is built and tested automatically, so integration problems show up within minutes.

Jenkins

Open-source, self-hosted automation server; pipelines defined in a Jenkinsfile kept with the code.

GitHub Actions

Workflows written in YAML under `.github/workflows`, run on GitHub-hosted or self-hosted runners.

ShopFast quality gate

On push or pull request: `npm ci`, install Chromium, `npm test`, upload the Playwright report.

Broken build first

A red build is fixed, or the change reverted, before any new work is merged.

Build freeze

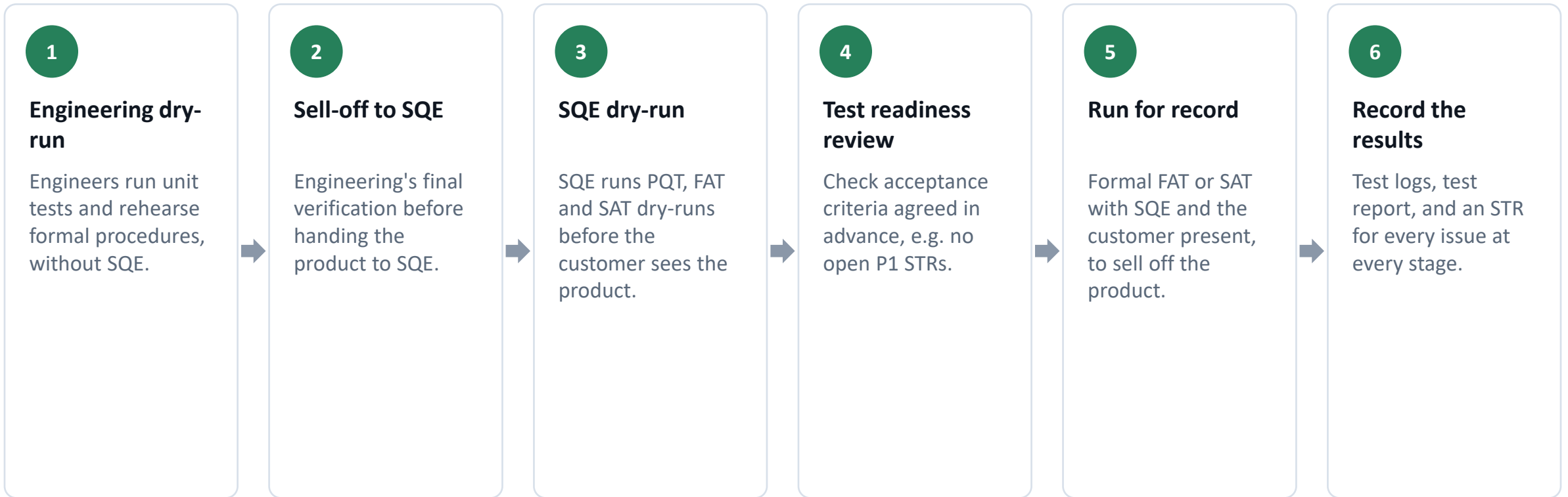
No code changes during formal tests. The build under test is identified and stays unchanged.

Dynamic analysis: learning from the running program

The execution-based counterpart to static analysis (reviews and tool checks without running code)

Technique	What it does	Example tools	ShopFast use
Monitoring run-time behaviour	Observes the running program: errors, memory, timing, requests	AddressSanitizer (C++), APM tools, request logs	One request log line per API call (FR-12)
Automated test case generation	A tool generates many inputs automatically	Fuzzers such as libFuzzer; property-based tools fast-check, Hypothesis	Random carts checked so total equals subtotal minus discount plus shipping
Coverage analysis	Measures which statements and branches the tests executed	gcov or lcov (C++), c8 or nyc (Node.js)	Find pricing branches that no test has reached
Assertions	Checks inside the code that fail loudly when an assumption breaks	assert() in C and C++, thrown errors in JavaScript	Assert every order status is one of the lifecycle states

Formal test runs in software quality control



The build stays frozen through the formal runs. FAT is the factory acceptance test, SAT the site acceptance test.

Goal-directed test types, clarified

Goal	Technique	What it means	ShopFast example
Security	Security testing	Broad: any test of security properties such as access control and authentication	Login lockout and order privacy rules in BR-07 behave as specified
Vulnerabilities	Penetration testing	Narrower: authorised, simulated attacks that try to exploit weaknesses	Tampered session header, injection attempts in the promo code field
Invalid inputs	Robustness testing	Copes with invalid or unexpected input without crashing or corrupting data	Quantity abc, malformed JSON body, very long name
Compatibility	Compatibility testing	Works across browsers, devices, operating systems and versions	Checkout in Chromium, Firefox and WebKit, and at phone width
Performance	Performance testing	Response time, throughput and resource use under expected and peak load	p95 checkout under 2 s with 50 concurrent users (REQ-CHK-06)

Makmal A, Pengujian API dengan [Postman](#) / API Testing with [Postman](#)

Sahkan API checkout ShopFast bercakap seperti dikontrakkan, tanpa menyentuh satu skrin UI pun. / Verify the checkout API behaves as contracted, without touching the UI.

1 Bina koleksi [Postman](#) untuk API checkout terus daripada spesifikasi [OpenAPI](#).
Build a [Postman](#) collection for the checkout API from the [OpenAPI](#) spec.

2 Tambah assertions: kod status, skema respons & masa respons.
Add assertions: status code, response schema & response time.

3 Jalankan koleksi tanpa UI guna [Newman](#) & baca laporannya.
Run the collection headless with [Newman](#) & read the report.

[Postman](#)

[Newman](#) (CLI)

[OpenAPI](#) Spec

[OpenAPI](#) Spec

Kontrak API checkout / API contract



[Postman](#) Collection

Bina permintaan + assertions / Build requests + assertions



[Newman](#) (headless run)

Jalankan tanpa UI dalam CLI / Run headless in CLI



Laporan / Report

Lulus / gagal, terperinci / Pass/fail, detailed

OUTPUT: Koleksi [Postman](#) lengkap dengan assertions, boleh dijalankan semula bila-bila masa. / A complete [Postman](#) collection with assertions, rerunnable any time.

Real run: Newman on the ShopFast API

Captured on the lab PC: npm run test:api in lab/shopfast

```
SQA-SHOT

→ 06 Checkout with valid details
  POST http://localhost:3000/api/checkout [201 Created, 783B, 2ms]
  ✓ status is 201
  ✓ order id starts with SF-

→ 07 Reject quantity 0
  POST http://localhost:3000/api/cart/items [400 Bad Request, 416B, 2ms]
  ✓ status is 400
  ✓ error code is VALIDATION_ERROR

→ 08 Reject unknown discount code
  POST http://localhost:3000/api/discounts/apply [422 Unprocessable Entity, 420B, 2ms]
  ✓ status is 422
  ✓ error code is INVALID_CODE
```

	executed	failed
iterations	1	0
requests	8	0
test-scripts	8	0
prerequest-scripts	8	0
assertions	16	0

```
total run duration: 694ms
total data received: 1.7kB (approx)
average response time: 8ms [min: 2ms, max: 46ms, s.d.: 14ms]

PS C:\Users\user\sqa-krisa-bengkel\lab\shopfast>
```

Makmal B, Automasi UI dengan [Playwright](#) / UI Automation with [Playwright](#)

Tulis skrip pertama yang berkelakuan seperti pengguna sebenar, dan buktikan ia menangkap kerosakan. / Write a script that behaves like a real user, and prove it catches breakage.

1

Tulis ujian [Playwright](#) (atau [Selenium](#) + pytest) pertama untuk log masuk to checkout.

Write your first [Playwright](#) (or [Selenium](#)+pytest) test for login to checkout.

2

Tambah assertion pada skrin pengesahan pesanan; pastikan ia stabil.

Add an assertion on the order-confirmation screen; make it stable.

3

Jalankan & lihat ia lulus, kemudian rosakkan app dan lihat ia gagal dengan jelas.

Run it and watch it pass, then break the app and watch it fail clearly.

[Playwright](#)

[Selenium](#)

[Cypress](#)

Login to Checkout Script

[Playwright](#) / [Selenium](#) + pytest



Assertion pada pengesahan

Order-confirmation screen



✓ RUN, LULUS / PASS

App stabil / App stable

✗ APP ROSAK / BROKEN

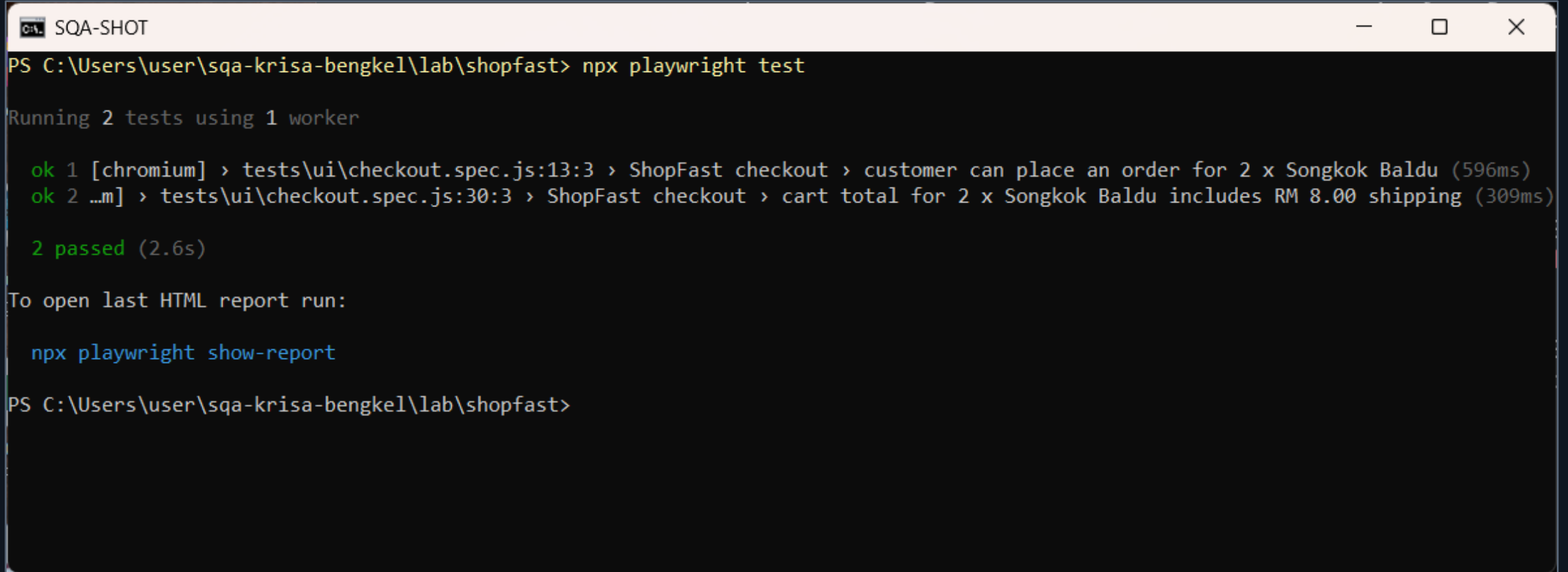
Ujian gagal jelas / Fails clearly

Bukti: ujian menangkap kerosakan sebenar / Proof the test catches real breakage

OUTPUT: Satu skrip automasi UI yang lulus, bukti ujian anda menguji sesuatu yang penting. / One passing UI automation script.

Real run: Playwright UI tests

Captured on the lab PC: npx playwright test in lab/shopfast



```
SQA-SHOT
PS C:\Users\user\sqa-krisa-bengkel\lab\shopfast> npx playwright test

Running 2 tests using 1 worker

ok 1 [chromium] > tests\ui\checkout.spec.js:13:3 > ShopFast checkout > customer can place an order for 2 x Songkok Baldu (596ms)
ok 2 ...m] > tests\ui\checkout.spec.js:30:3 > ShopFast checkout > cart total for 2 x Songkok Baldu includes RM 8.00 shipping (309ms)

2 passed (2.6s)

To open last HTML report run:

npx playwright show-report

PS C:\Users\user\sqa-krisa-bengkel\lab\shopfast>
```

Sampel alatan rujukan, semua bertaraf industri / Sample tool references

Boleh ditukar mengikut stack pasukan anda, konsepnya yang penting, bukan jenama. / Swappable to match your stack, the concept matters, not the brand.

[Postman](#) + [Newman](#)

Bina & jalankan koleksi ujian API tanpa UI.
Build & run headless API test collections.

[Playwright](#)

Automasi pelayar rentas-platform, stabil & pantas.
Cross-browser automation, stable & fast.

[Selenium](#) / [Cypress](#)

Alternatif automasi UI popular, konsep sama.
Popular UI automation alternatives.

[JMeter](#) / [k6](#) (demo)

Simulasi beban & tekanan pada API checkout.
Simulate load & stress on the checkout API.

[OWASP ZAP](#) (demo)

Imbasan kelemahan keselamatan automatik.
Automated security vulnerability scanning.

[GitHub Actions](#) (pratonton)

Rumah untuk Quality Gate automasi di Modul 8.
Home for the automation Quality Gate (Module 8).

Apa yang anda bawa keluar dari modul ini / What you carry out of this module

Dua artifak konkrit, bukan nota, tetapi kod & koleksi yang boleh dijalankan semula. / Two concrete, rerunnable artifacts.

Koleksi [Postman](#) + skrip UI [Postman](#) collection + UI script

Koleksi [Postman](#) dengan ujian penuh, dan satu skrip automasi UI yang lulus, kedua-duanya boleh dijalankan semula pada bila-bila masa.

A [Postman](#) collection with full tests, and one passing UI script, both rerunnable at will.

Menyokong Laporan Ujian Sistem Feeds the System Test Report

Bukti ujian pagi ini menjadi input kepada Laporan Ujian Sistem & keterjeakan RTM yang disemak di Modul 7 (Audit).

This evidence feeds the System Test Report & RTM traceability reviewed in Module 7 (Audit).

Selaras KRISA 2.0 (JDN) / KRISA 2.0 Alignment (JDN)

[Bab 5](#) · Pembangunan, pengujian sistem menghasilkan Laporan Ujian Sistem. [Bab 8](#) · Penyelarasan DevOps, automasi ujian ialah asas kepada pipeline kualiti berterusan.

[Ch.5](#) Development, system testing produces the System Test Report. [Ch.8](#) DevOps Alignment, test automation is the foundation of the continuous quality pipeline.

Singkatan dalam modul ini (1 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
AI	Artificial Intelligence	Kecerdasan Buatan. Perisian yang boleh menjawab soalan dan menulis teks atau kod berdasarkan pola daripada data yang banyak. <i>Software that can answer questions and write text or code by learning patterns from large amounts of data.</i>
API	Application Programming Interface	Pintu yang membolehkan satu program meminta data atau tindakan daripada program lain tanpa melalui skrin. <i>A doorway that lets one program ask another program for data or an action, without using a screen.</i>
APM	Application Performance Monitoring	Alat yang memantau sistem yang sedang beroperasi dan menunjukkan kelajuan serta kesihatannya. <i>Tools that watch a live system and show how fast and healthy it is.</i>
BR	Business Rule	Peraturan yang ditetapkan oleh bisnes, contohnya siapa boleh melihat pesanan. Digunakan dalam ID seperti BR-07. <i>A rule set by the business, for example who may view an order. Used in IDs such as BR-07.</i>
BVA	Boundary Value Analysis	Analisis Nilai Sempadan. Menguji di hujung julat nilai seperti 0, 1, 10 dan 11 kerana kesilapan kerap berlaku di situ. <i>Testing at the edges of the allowed values, such as 0, 1, 10 and 11, because mistakes often hide there.</i>
CI	Continuous Integration, or Configuration Item	Dua maksud. Dalam DevOps, setiap perubahan kod dibina dan diuji secara automatik. Dalam pengurusan konfigurasi, ia ialah item terkawal seperti dokumen atau fail kod. <i>Two meanings. In DevOps, every code change is built and tested automatically. In configuration management, it is any controlled item such as a document or code file.</i>
CI/CD	Continuous Integration and Continuous Delivery	Saluran automatik yang membina, menguji dan menyediakan setiap perubahan untuk dikeluarkan. <i>An automated pipeline that builds, tests and prepares every change for release.</i>

Singkatan dalam modul ini (2 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
CLI	Command Line Interface	Antara muka baris arahan, iaitu tetingkap teks untuk menaip arahan dan bukannya klik butang. <i>A text window where you type commands instead of clicking buttons.</i>
DB	Database	Pangkalan data, iaitu tempat sistem menyimpan data seperti pesanan dan pelanggan. <i>Where the system keeps its data, such as orders and customers.</i>
E2E	End-to-End	Ujian yang menjalankan perjalanan pengguna yang lengkap melalui skrin dan perkhidmatan sebenar. <i>A test that runs a full user journey through the real screens and services.</i>
EP	Equivalence Partitioning	Pembahagian Kesetaraan. Mengumpulkan input yang berkelakuan sama, kemudian menguji satu nilai daripada setiap kumpulan. <i>Grouping inputs that behave the same way, then testing one value from each group.</i>
FAT	Factory Acceptance Test	Ujian Penerimaan Kilang. Ujian formal bersama pelanggan sebelum sistem dipasang di tapak sebenar. <i>The formal test with the customer before the system is installed at the live site.</i>
FR	Functional Requirement	Keperluan Fungsian, iaitu perkara yang mesti dilakukan oleh sistem. Digunakan dalam ID seperti FR-03. <i>Something the system must do. Used in IDs such as FR-03.</i>
ID	Identifier	Pengecam, iaitu label unik seperti REQ-CHK-01 supaya setiap item boleh dijejak. <i>A unique label, such as REQ-CHK-01, so that each item can be traced.</i>

Singkatan dalam modul ini (3 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
JDN	Jabatan Digital Negara	Jabatan kerajaan Malaysia yang menerbitkan KRISA dan PPrISA. <i>The Malaysian government department that publishes KRISA and PPrISA.</i>
JS	JavaScript	Bahasa pengaturcaraan yang digunakan oleh laman web dan aplikasi makmal ShopFast. <i>The programming language used by web pages and by the ShopFast lab app.</i>
JSON	JavaScript Object Notation	Format teks untuk data berstruktur, biasa digunakan oleh API. <i>A plain text format for structured data, often used by APIs.</i>
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	Buku Panduan Kejuruteraan Sistem Aplikasi Sektor Awam. KRISAv2 ialah edisi beta 2026. <i>The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.</i>
MIT	Massachusetts Institute of Technology	Universiti di Amerika Syarikat. Sumber nota kursus Leveson. <i>A university in the United States. Source of the Leveson course notes.</i>
OWASP	Open Worldwide Application Security Project	Komuniti yang menerbitkan panduan keselamatan web percuma seperti OWASP Top 10. <i>A community that publishes free web security guides such as the OWASP Top 10.</i>

Singkatan dalam modul ini (4 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
PAT	Provisional Acceptance Test	Ujian Penerimaan Sementara dalam KRISA sebelum sistem diterima secara sementara, termasuk semakan prestasi. <i>The KRISA acceptance test done before provisional acceptance of the system, including performance checks.</i>
PC	Personal Computer	Komputer peribadi, iaitu komputer makmal yang anda gunakan. <i>The lab computer you are working on.</i>
PDF	Portable Document Format	Format dokumen yang kelihatan sama pada semua komputer. <i>A document file that looks the same on every computer.</i>
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	Panduan Pengurusan Projek ICT Sektor Awam. Kod seperti PPrISA05 ialah templatnya. <i>The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.</i>
PQT	Preliminary Qualification Test	Ujian formal yang menunjukkan setiap bahagian perisian memenuhi keperluan sebelum ujian sistem penuh. <i>A formal test showing that each software part meets its requirements before full system testing.</i>
PR	Pull Request	Permintaan dalam GitHub untuk menyemak dan menggabungkan perubahan kod ke dalam cabang utama. <i>A request on GitHub to review and merge a code change into the main branch.</i>

Singkatan dalam modul ini (5 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
PS	PowerShell prompt	Apabila slaid menunjukkan PS>, taip arahan itu dalam Windows PowerShell. <i>When a slide shows PS>, type that command in Windows PowerShell.</i>
REQ	Requirement	Keperluan. Awalan dalam ID keperluan seperti REQ-CHK-01. CHK bermaksud checkout. <i>Prefix in requirement IDs such as REQ-CHK-01. CHK stands for checkout.</i>
RM	Ringgit Malaysia	Mata wang Malaysia. <i>Malaysian currency.</i>
RTM	Requirements Traceability Matrix	Matriks Kebolehkesanan Keperluan. Jadual yang memautkan setiap keperluan kepada reka bentuk, ujian dan kecacatan supaya tiada yang tertinggal. <i>A table linking each requirement to its design, tests and defects, so nothing is missed.</i>
SAT	Site Acceptance Test	Ujian Penerimaan Tapak. Ujian penerimaan formal di tapak sebenar. <i>The formal acceptance test carried out at the live site.</i>
SIT	System Integration Test	Ujian Integrasi Sistem, iaitu menguji bahawa komponen sistem dan sistem lain yang bersambung berfungsi dengan betul bersama-sama. <i>Testing that the parts of a system, and the systems it connects to, work correctly together.</i>

Singkatan dalam modul ini (6 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
SQA	Software Quality Assurance	Jaminan Kualiti Perisian, iaitu semua kerja terancang yang memberi keyakinan bahawa perisian memenuhi keperluan kualiti. <i>All the planned work that gives confidence the software meets its quality requirements.</i>
SQE	Software Quality Engineer	Jurutera Kualiti Perisian, iaitu individu yang menjalankan semakan kualiti dan audit serta menyaksikan ujian formal. <i>The person who runs quality checks and audits and witnesses formal tests.</i>
SRS	Spesifikasi Keperluan Sistem	Spesifikasi Keperluan Sistem, iaitu perkara yang mesti dilakukan oleh sistem. Serahan KRISA D03. <i>The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.</i>
STR	Software Trouble Report	Laporan Masalah Perisian, iaitu borang yang merekod pepijat dari saat ditemui hingga ditutup. <i>The form that records a bug from the moment it is found until it is closed.</i>
TC	Test Case	Kes Ujian. Awalan dalam ID kes ujian seperti TC-CHK-004. <i>Prefix in test case IDs such as TC-CHK-004.</i>
TDD	Test-Driven Development	Pembangunan Dipacu Ujian. Tulis ujian yang gagal dahulu, kemudian tulis kod secukupnya supaya ujian lulus. <i>Write a failing test first, then write just enough code to make it pass.</i>

Singkatan dalam modul ini (7 daripada 7)

Setiap singkatan dalam slaid ini, diterangkan dengan bahasa mudah. / Every short form on these slides, in plain words.

Singkatan	Maksud penuh / Stands for	Maksud mudah / In plain words
UAT	User Acceptance Test	Ujian Penerimaan Pengguna. Pengguna mencuba sistem untuk memastikan ia memenuhi keperluan sebelum menerimanya. <i>Users try the system to confirm it meets their needs before they accept it.</i>
UI	User Interface	Antara muka pengguna, iaitu skrin, butang dan borang yang digunakan orang. <i>The screens, buttons and forms that people use.</i>
WCAG	Web Content Accessibility Guidelines	Garis panduan supaya laman web boleh digunakan oleh orang kurang upaya. <i>Rules that make websites usable by people with disabilities.</i>
xUnit	xUnit test frameworks	Keluarga alat ujian unit seperti JUnit, NUnit dan Google Test yang berkongsi struktur yang sama. <i>The family of unit test tools, such as JUnit, NUnit and Google Test, that share the same structure.</i>
YAML	YAML Ain't Markup Language	Format teks yang mudah dibaca untuk fail tetapan seperti aliran kerja GitHub Actions. <i>A readable text format for settings files, such as GitHub Actions workflows.</i>
ZAP	Zed Attack Proxy	Alat percuma OWASP yang mengimbas aplikasi web untuk mencari kelemahan keselamatan. <i>A free OWASP tool that scans web applications for security weaknesses.</i>

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

- Bab 5, 5.7 Pengujian Sistem F4.3
page 33 · KRISAv2 Bab 5 Fasa Pembangunan
- Bab 5, Langkah 5 Laporan Ujian Sistem (Jadual 5.6)
page 39 · KRISAv2 Bab 5 Fasa Pembangunan
- Bab 5, 5.6.7 Amalan terbaik (Git, piawaian kod)
page 28 · KRISAv2 Bab 5 Fasa Pembangunan
- Bab 6, 6.9 PAT (prestasi, bebanan, keselamatan)
page 23 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan
- Bab 7, 7.10 Persekitaran Pembangunan (Dev, Staging, Prod)
page 19 · KRISAv2 Bab 7 Fasa Pelaksanaan
- Bab 8, 8.2.2 Pendekatan DevOps dan CI/CD
page 3 · KRISAv2 Bab 8 Penyelarasan DevOps

TEMPLATES TO USE

- D10
Dokumentasi Kod Sumber

WORDPDF
- D11
Laporan Ujian Sistem

WORDPDF

Anda kini tahu *apa yang rusak*. *Seterusnya: uruskannya. / Next: govern it.*

- Empat lensa ujian fungsian: Smoke, Sanity, Regression, Exploratory.
Four functional test lenses.
- Bukan fungsian membuktikan sistem pantas, selamat & boleh dicapai semua.
Non-functional testing proves speed, safety & accessibility.
- Piramid Automasi: banyak Unit, sederhana API, sedikit UI.
Automation Pyramid: lots of Unit, moderate API, few UI.
- Dua hasil kerja sebenar: koleksi Postman + 1 skrip UI yang lulus.
Two real deliverables: Postman collection + 1 passing UI script.

SETERUSNYA / NEXT UP

Modul 6, Pengurusan Pepijat & Metrik Kualiti. Bawa apa yang anda temui pagi ini; kita akan log, triage & ukur ia.
Module 6, Bug Management & Quality Metrics. Bring what you found; we'll log, triage & measure it.

M05 · W05 · 15 MINUTES · Hari 2, 10:10 hingga 10:25

Hands-On SQA-AI Assistant Workshop

AI-generated UI and API tests that find a real bug

Ask the AI for a Playwright test for the quantity boundary, save and run it against ShopFast, read the failure, then generate API tests from the OpenAPI contract.

Minta AI menjana ujian Playwright untuk sempadan kuantiti, simpan dan jalankan pada ShopFast, baca kegagalan, kemudian jana ujian API daripada kontrak OpenAPI.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W05\)](#)[Setup guide W00](#)[opencode docs](#)

AI-generated UI and API tests that find a real bug

STEPS

- 1 Run `/gen-ui-test` for TC-CHK-004 (quantity 10)
- 2 Save the code block as `tests\ui\m05-senario.spec.js`
- 3 Run it with `npx playwright test` and read the result
- 4 Run `/gen-api-tests` for the same requirements

OUTPUT TO SAVE

`outputs/m05-ui-test.md`, `shopfast/tests/ui/m05-senario.spec.js`, `outputs/m05-api-tests.md`

TYPE THIS (`opencode>` in `opencode`, `PS>` in PowerShell)

```
opencode> /gen-ui-test TC-CHK-004 tambah P002 kuantiti 10 ke troli dan  
semak troli menerima kuantiti 10 (REQ-CHK-01)
```

```
PS> cd shopfast ; npx playwright test tests/ui/m05-senario.spec.js
```

```
opencode> /gen-api-tests REQ-CHK-01 REQ-CHK-03
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- The test selects elements by data-testid that really exist
- A failing test is a finding: reproduce it by hand before logging a bug
- Generated tests are reviewed like code before they are committed

[Open step-by-step guide](#)

[Module hub](#)

Pembantu AI dalam SQA: AI mendraf, manusia bertanggungjawab

AI assistant in SQA: AI drafts, humans remain accountable

Aliran kerja setiap bengkel Hands-On SQA-AI Assistant Workshop menggunakan opencode

1

Konteks

@SRS, @templat,
AGENTS.md

2

Prompt

arahan /review-srs

3

Draf AI

opencode + model

4

Semak manusia

banding dengan templat
KRISA/PPrISA

5

Simpan bukti

outputs/ + git commit

Guna AI untuk

Draf pelan dan laporan, cadang kes ujian
EP/BVA, jana skrip ujian, ringkaskan log, cari
jurang RTM.

Jangan

Tampal data sebenar agensi, terima output
tanpa semak, biar AI reka ID keperluan, kongsi
sesi awam.

Bukti audit

Simpan prompt, output dan semakan dalam git
supaya jejak audit lengkap (Modul 7).